In [52]:
```
from factor_diffusion_train import FACTOR_NAMES, PREFIX, MODE
from factor_model import FactorModel, load_model
from factor_diffusion_sample import NUM_GENERATE
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
```

In [53]:
```
model = load_model(PREFIX)
test_model = load_model(f"{PREFIX}/test")
print(PREFIX)
```

```
model/regression
```

Factor Level Evaluation

In [54]:
```
sampled = np.load(f"{PREFIX}/samples/factor_{MODE}_{NUM_GENERATE}.npy")
print(f"{PREFIX}/samples/factor_{MODE}_{NUM_GENERATE}.npy")
```

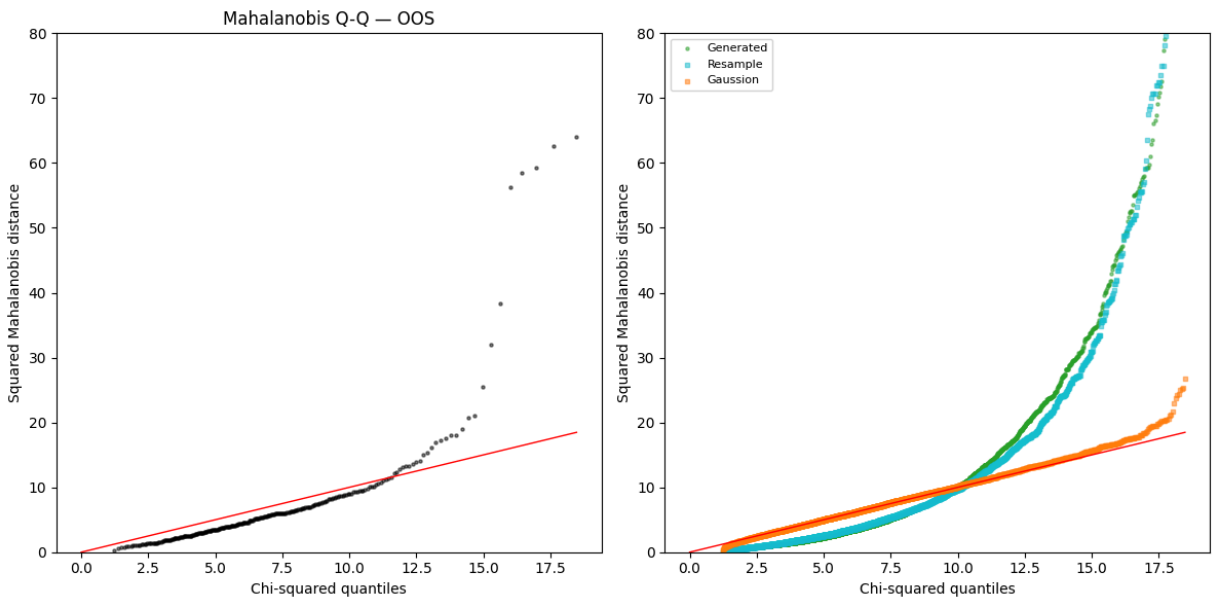
```
model/regression/samples/factor_DLPM_4096.npy
```

In [55]:
```
train = model.F
rng = np.random.default_rng(42)
cov = np.cov(train.drop(columns=['alpha']).values, rowvar=False)
sampled_gaussian = rng.multivariate_normal(mean=train.drop(columns=['alpha']

OOS = test_model.F.drop(columns=['alpha']).values
row_idx = rng.choice(train.shape[0], size=len(sampled), replace=True)
baselines = [("Resample", train.iloc[row_idx].drop(columns=['alpha']).values
print(f"OOS: {OOS.shape}  sampled: {sampled.shape}  Gaussian: {sampled_gauss
```

```
OOS: (260, 7)  sampled: (4096, 7)  Gaussian: (4096, 7)
```

In [56]:
```
# 2.0 Mahalanobis check
def mahal_d2(X):
    mean = X.mean(axis=0)
    cov  = np.cov(X, rowvar=False)
    diff = X - mean
    return (diff @ np.linalg.pinv(cov) * diff).sum(axis=1)

def plot_mahal(ax, X, label, color='steelblue', marker='o', size=5, alpha=0.
    p  = X.shape[1]
    d2 = mahal_d2(X)
    qs = stats.chi2.ppf(np.linspace(0.01, 0.99, len(d2)), df=p)
    ax.scatter(qs, np.sort(d2), s=size, alpha=alpha, color=color,
               marker=marker, label=label)
    return qs.max()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# Left panel: OOS
q_max = plot_mahal(ax1, OOS, 'OOS', color='black')
ax1.plot([0, q_max], [0, q_max], color='red', linewidth=1)
ax1.set_ylim(0, 80)
```

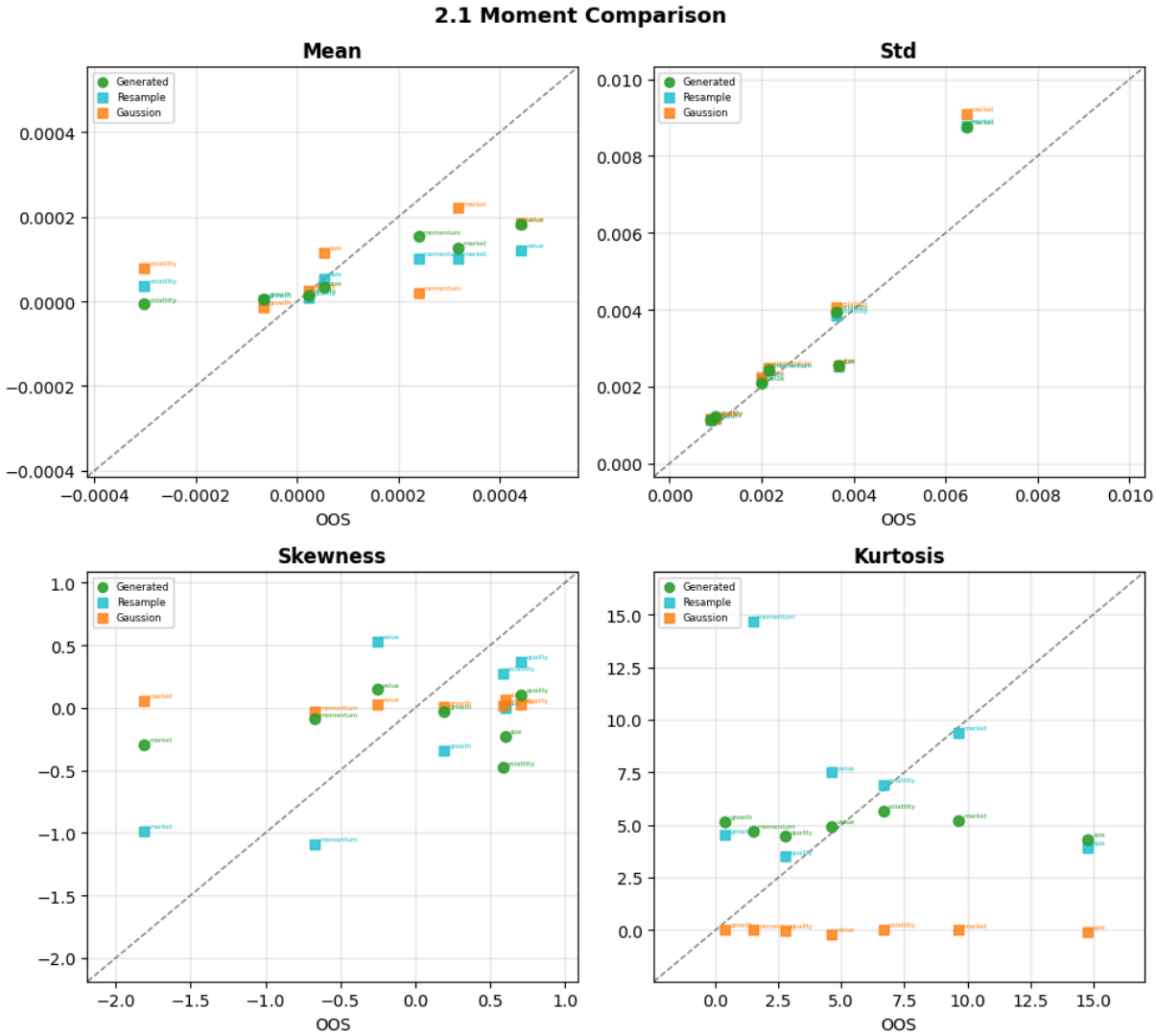
```
ax1.set_xlabel('Chi-squared quantiles'); ax1.set_ylabel('Squared Mahalanobis
ax1.set_title('Mahalanobis Q-Q — OOS')

# Right panel: Generated + Baselines overlaid
q_max2 = plot_mahal(ax2, sampled, 'Generated', color='#2ca02c')
for (bname, bdata, bc) in baselines:
    q_max2 = max(q_max2, plot_mahal(ax2, bdata, bname, color=bc, marker='s')
ax2.plot([0, q_max2], [0, q_max2], color='red', linewidth=1)
ax2.set_ylim(0, 80)
ax2.set_xlabel('Chi-squared quantiles'); ax2.set_ylabel('Squared Mahalanobis
ax2.legend(fontsize=8)
plt.tight_layout()
plt.show()
```



In [57]:

```
# 2.1 Moment Comparison
def compute_moments(data):
    return [np.mean(data, axis=0), np.std(data, axis=0),
            stats.skew(data, axis=0), stats.kurtosis(data, axis=0)]


moment_names = ['Mean', 'Std', 'Skewness', 'Kurtosis']
m_OOS    = compute_moments(OOS)
m_gen    = compute_moments(sampled)
m_bases  = [(name, compute_moments(data), c) for name, data, c in baselines]

ANNOT_FS = 4
LEGEND_FS = 6
PT_SIZE   = 40

fig, axes = plt.subplots(2, 2, figsize=(10, 9))
for idx, (ax, mname) in enumerate(zip(axes.flatten(), moment_names)):
    OOS_m = m_OOS[idx]
    gen_m = m_gen[idx]
    all_y = [gen_m] + [bm[idx] for _, bm, _ in m_bases]

    lim = [min(OOS_m.min(), min(y.min() for y in all_y)),
           max(OOS_m.max(), max(y.max() for y in all_y))]
```

```
    pad = (lim[1] - lim[0]) * 0.15 or 0.01
    lim = [lim[0] - pad, lim[1] + pad]

    ax.plot(lim, lim, 'k--', lw=1, alpha=0.5)
    ax.scatter(OOS_m, gen_m, color='#2ca02c', s=PT_SIZE, marker='o', zorder=
               label='Generated', alpha=0.9)
    for i, fn in enumerate(FACTOR_NAMES):
        ax.annotate(fn, (OOS_m[i], gen_m[i]), fontsize=ANNOT_FS,
                    xytext=(3, 2), textcoords='offset points', color='#2ca02
    for (bname, bm, bc) in m_bases:
        ax.scatter(OOS_m, bm[idx], color=bc, s=PT_SIZE, marker='s', zorder=3
                   label=bname, alpha=0.8)
        for i, fn in enumerate(FACTOR_NAMES):
            ax.annotate(fn, (OOS_m[i], bm[idx][i]), fontsize=ANNOT_FS,
                        xytext=(3, 2), textcoords='offset points', color=bc)

    ax.set_xlim(lim); ax.set_ylim(lim)
    ax.set_xlabel('OOS')
    ax.set_title(mname, fontweight='bold')
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=LEGEND_FS, markerscale=0.9, handlelength=1.2)

fig.suptitle('2.1 Moment Comparison', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

# ── MAE summary — mean (std) across factors ──────────────────────────
def fmt_mae(errs):
    return f"{np.mean(errs):.4f} ({np.std(errs):.4f})"

col_w = 22
header = f"{'Moment':<12}" + f"{'Generated':<{col_w}}" + \
         "".join(f"{bname:<{col_w}}" for bname, _, _ in m_bases)
print("MAE across factors — mean (std)  |Generated/Baseline - OOS|:")
print(header)
print("-" * len(header))
for idx, mname in enumerate(moment_names):
    row = f"{mname:<12}{fmt_mae(np.abs(m_gen[idx] - m_OOS[idx])):<{col_w}}"
    for _, bm, _ in m_bases:
        row += f"{fmt_mae(np.abs(bm[idx] - m_OOS[idx])):<{col_w}}"
    print(row)
```

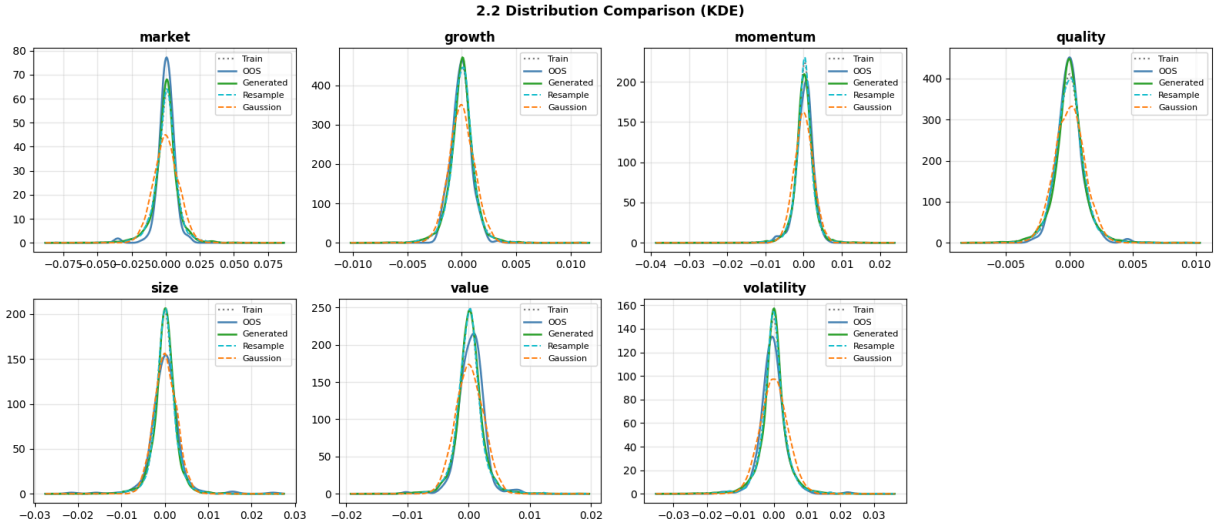



```
MAE across factors — mean (std)  |Generated/Baseline − OOS|:
Moment      Generated             Resample              Gaussion
-------------------------------------------------------------------------------
--
Mean        0.0001 (0.0001)       0.0002 (0.0001)       0.0002 (0.0001)
Std         0.0007 (0.0007)       0.0007 (0.0008)       0.0007 (0.0008)
Skewness    0.7443 (0.4028)       0.5458 (0.1893)       0.6811 (0.5136)
Kurtosis    3.7032 (3.1796)       4.6070 (4.9193)       5.8091 (4.7249)
```

In [64]:

```python
# 2.2 Distribution Comparison (KDE)
from scipy.stats import gaussian_kde, wasserstein_distance

n_factors = len(FACTOR_NAMES)
fig, axes = plt.subplots(2, 4, figsize=(16, 7))

for ax, name, i in zip(axes.flatten(), FACTOR_NAMES, range(n_factors)):
    OOS_vals    = OOS[:, i]
    gen_vals    = sampled[:, i]
    train_vals  = train.drop(columns=['alpha']).values[:, i]

    all_vals = np.concatenate([OOS_vals, gen_vals, train_vals] +
                              [bdata[:, i] for _, bdata, _ in baselines])
    x_min, x_max = all_vals.min(), all_vals.max()
    pad = (x_max - x_min) * 0.05
```

```
    x = np.linspace(x_min - pad, x_max + pad, 300)

    for vals, label, color, ls, lw in [
        (train_vals, 'Train',     '#7f7f7f', ':',  1.6),
        (OOS_vals,   'OOS',       'steelblue', '-', 1.8),
        (gen_vals,   'Generated', '#2ca02c',     '-', 1.8),
    ]:
        ax.plot(x, gaussian_kde(vals)(x), label=label, color=color, lw=lw, l

    for (bname, bdata, bc) in baselines:
        ax.plot(x, gaussian_kde(bdata[:, i])(x),
                label=bname, color=bc, lw=1.4, ls='--')

    ax.set_title(name, fontweight='bold')
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)

for ax in axes.flatten()[n_factors:]:
    ax.set_visible(False)

fig.suptitle('2.2 Distribution Comparison (KDE)', fontsize=13, fontweight='b
plt.tight_layout()
plt.show()

col_w = 14
header = f"{'Factor':<14}" + f"{'Generated':<{col_w}}" + \
         "".join(f"{bname:<{col_w}}" for bname, _, _ in baselines)
print("Wasserstein-1 Distance (vs OOS per factor):")
print(header)
print("-" * len(header))
for i, name in enumerate(FACTOR_NAMES):
    w1_gen = wasserstein_distance(OOS[:, i], sampled[:, i])
    row = f"{name:<14}{w1_gen:<{col_w}.4f}"
    for _, bdata, _ in baselines:
        row += f"{wasserstein_distance(OOS[:, i], bdata[:, i]):<{col_w}.4f}"
    print(row)
avg_gen = np.mean([wasserstein_distance(OOS[:, i], sampled[:, i]) for i in r
avgs_b  = [np.mean([wasserstein_distance(OOS[:, i], bdata[:, i]) for i in ra
           for _, bdata, _ in baselines]
print(row)
```

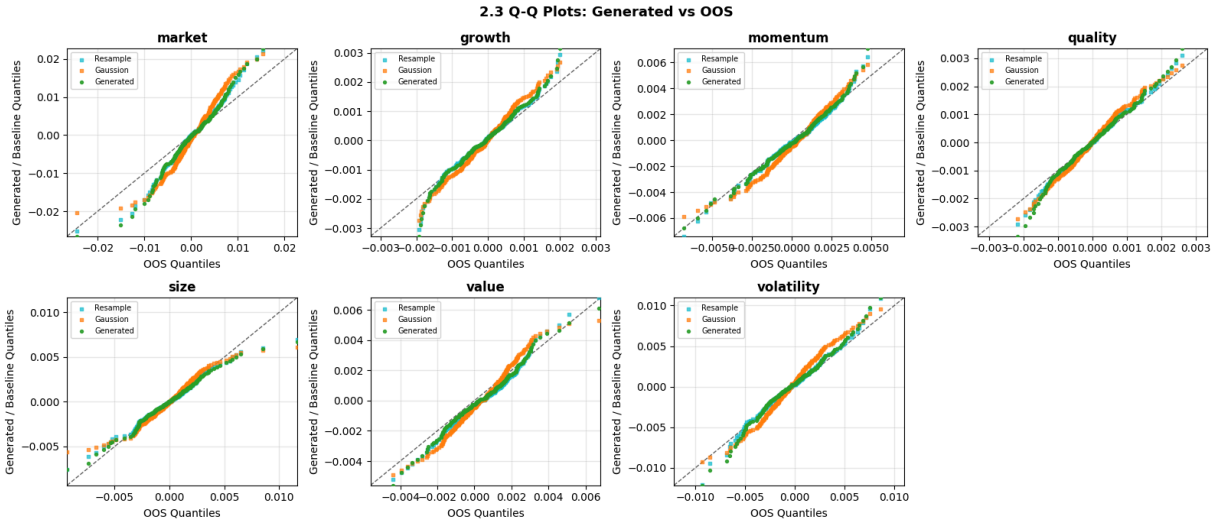



```
Wasserstein-1 Distance (vs OOS per factor):
Factor          Generated     Resample      Gaussion
-------------------------------------------------------
market          0.0017        0.0018        0.0032
growth          0.0002        0.0002        0.0002
momentum        0.0002        0.0003        0.0005
quality         0.0001        0.0001        0.0002
size            0.0004        0.0004        0.0005
value           0.0003        0.0004        0.0005
volatility      0.0005        0.0004        0.0009
volatility      0.0005        0.0004        0.0009
```

In [59]:

```
# 2.3 Quantile-Quantile (Q-Q) Plots
n_quantiles = 200
probs = np.linspace(0.01, 0.99, n_quantiles)
n_factors = len(FACTOR_NAMES)

fig, axes = plt.subplots(2, 4, figsize=(16, 7))

for ax, name, i in zip(axes.flatten(), FACTOR_NAMES, range(n_factors)):
    q_OOS  = np.quantile(OOS[:, i],      probs)
    q_gen  = np.quantile(sampled[:, i],  probs)

    all_q = [q_OOS, q_gen] + [np.quantile(bdata[:, i], probs) for _, bdata,
    lim = [min(q.min() for q in all_q), max(q.max() for q in all_q)]

    for (bname, bdata, bc) in baselines:
        q_base = np.quantile(bdata[:, i], probs)
        ax.scatter(q_OOS, q_base, s=8, color=bc, zorder=3,
                   label=bname, alpha=0.7, marker='s')

    ax.scatter(q_OOS, q_gen, s=8, color='#2ca02c', zorder=4, label='Generate

    ax.plot(lim, lim, 'k--', lw=1, alpha=0.6)
    ax.set_xlim(lim); ax.set_ylim(lim)
    ax.set_title(name, fontweight='bold')
    ax.set_xlabel('OOS Quantiles')
    ax.set_ylabel('Generated / Baseline Quantiles')
    ax.legend(fontsize=7)
    ax.grid(True, alpha=0.3)
```

```
# hide unused subplots
for ax in axes.flatten()[n_factors:]:
    ax.set_visible(False)

fig.suptitle('2.3 Q-Q Plots: Generated vs OOS', fontsize=13, fontweight='bol
plt.tight_layout()
plt.show()
```



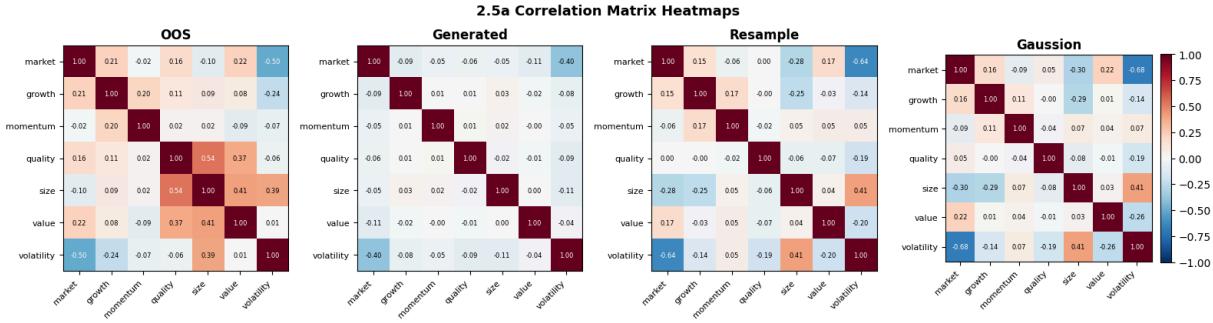
In [60]:

```
# 2.4 Kolmogorov-Smirnov Test
from scipy.stats import ks_2samp

P_THRESHOLD = 0.05
n_eff = len(OOS) * len(sampled) / (len(OOS) + len(sampled))

ks_gen   = [ks_2samp(OOS[:, i], sampled[:, i]).pvalue for i in range(len(FAC
ks_bases = [(bname, [ks_2samp(OOS[:, i], bdata[:, i]).pvalue
                     for i in range(len(FACTOR_NAMES))])
            for bname, bdata, _ in baselines]

def fmt(p): return f"{p:.4f} {'pass' if p > P_THRESHOLD else 'fail'}"

col_w = 16
header = f"{'Factor':<18} {'Generated':<{col_w}}" + \
         "".join(f"{bname:<{col_w}}" for bname, _ in ks_bases)
print(f"KS p-value (threshold p>{P_THRESHOLD})   n_eff={n_eff:.0f}")
print(header)
print("-" * len(header))
for j, name in enumerate(FACTOR_NAMES):
    row = f"{name:<18} {fmt(ks_gen[j]):<{col_w}}"
    for _, bks in ks_bases:
        row += f"{fmt(bks[j]):<{col_w}}"
    print(row)
```

```
KS p-value (threshold p>0.05)   n_eff=244
Factor              Generated       Resample        Gaussion
---------------------------------------------------------------------
market              0.0431 fail     0.0095 fail     0.0000 fail
growth              0.2697 pass     0.1973 pass     0.0031 fail
momentum            0.1174 pass     0.0395 fail     0.0070 fail
quality             0.5779 pass     0.4977 pass     0.0074 fail
size                0.4118 pass     0.5710 pass     0.3447 pass
value               0.0032 fail     0.0002 fail     0.0032 fail
volatility          0.0314 fail     0.0814 pass     0.0006 fail
```

In [61]:

```
from matplotlib.colors import TwoSlopeNorm
from metrics.statistic import compute_cov

def corr_mat(data):
    return np.corrcoef(data, rowvar=False)

corr_OOS  = corr_mat(OOS)
corr_gen  = corr_mat(sampled)
corr_base = [(bname, corr_mat(bdata), bc) for bname, bdata, bc in baselines]

all_corrs = [('OOS', corr_OOS, 'steelblue'),
             ('Generated', corr_gen, '#2ca02c')] + corr_base

# ── 2.5a Correlation Matrix Heatmaps ──────────────────────────────
n = len(all_corrs)
fig, axes = plt.subplots(1, n, figsize=(4 * n, 4.2))
norm = TwoSlopeNorm(vmin=-1, vcenter=0, vmax=1)

for ax, (name, corr, _) in zip(axes, all_corrs):
    im = ax.imshow(corr, cmap='RdBu_r', norm=norm)
    ax.set_xticks(range(len(FACTOR_NAMES)))
    ax.set_yticks(range(len(FACTOR_NAMES)))
    ax.set_xticklabels(FACTOR_NAMES, rotation=45, ha='right', fontsize=8)
    ax.set_yticklabels(FACTOR_NAMES, fontsize=8)
    ax.set_title(name, fontweight='bold')
    for ii in range(len(FACTOR_NAMES)):
        for jj in range(len(FACTOR_NAMES)):
            ax.text(jj, ii, f'{corr[ii, jj]:.2f}', ha='center', va='center',
                    fontsize=6, color='white' if abs(corr[ii, jj]) > 0.5 els

fig.colorbar(im, ax=axes[-1], fraction=0.046, pad=0.04)
fig.suptitle('2.5a Correlation Matrix Heatmaps', fontsize=13, fontweight='bo
plt.tight_layout()
plt.show()

# ── 2.5b Covariance Distance (relative Frobenius) ──────────────────
print("Relative Frobenius Distance of Covariance Matrix (||Cov_x - Cov_OOS||
print(f"  {'Generated':<16} {compute_cov(sampled, OOS):.4f}")
for bname, bdata, _ in baselines:
    print(f"  {bname:<16} {compute_cov(bdata, OOS):.4f}")
```



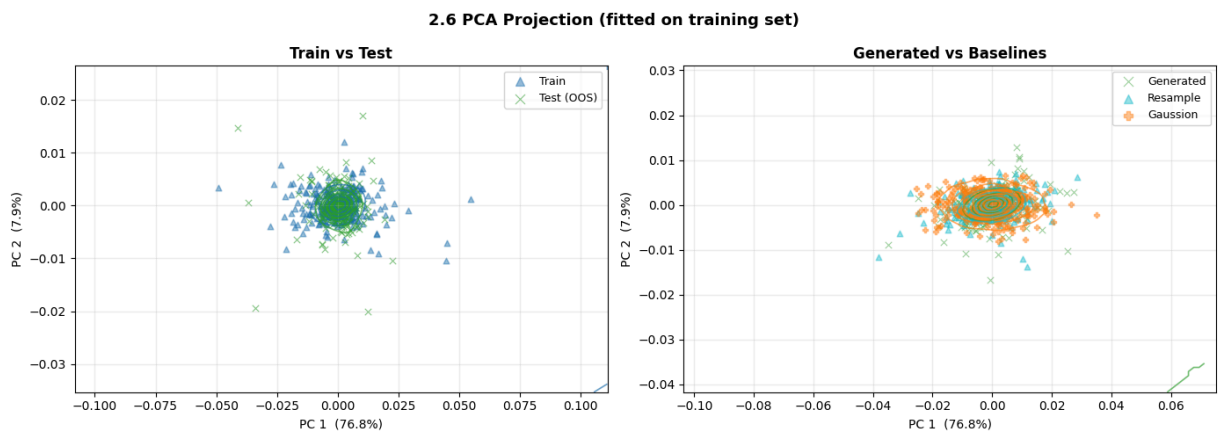

```
Relative Frobenius Distance of Covariance Matrix (||Cov_x - Cov_OOS||_F / ||
Cov_OOS||_F):
  Generated        0.7601
  Resample         0.7911
  Gaussion         0.9316
```

Dimensionality Reduction

In [62]:

```
# 2.6 PCA Projection
from sklearn.decomposition import PCA
from scipy.stats import gaussian_kde

train_vals = train.drop(columns=['alpha']).values

pca = PCA(n_components=2)
pca.fit(train_vals)

train_pc = pca.transform(train_vals)
OOS_pc   = pca.transform(OOS)
gen_pc   = pca.transform(sampled)
base_pcs = [(bname, pca.transform(bdata), bc) for bname, bdata, bc in baseli

xlab = f"PC 1  ({pca.explained_variance_ratio_[0]*100:.1f}%)"
ylab = f"PC 2  ({pca.explained_variance_ratio_[1]*100:.1f}%)"

N_SUB   = 300
C1, C2  = '#1a6faf', '#2ca02c'
MARKERS = ['^', 'P', 'D', 's']

def _sub(arr, n=N_SUB, seed=0):
    idx = np.random.default_rng(seed).choice(len(arr), size=min(n, len(arr))
    return arr[idx]

def _contour(ax, pc, color, levels=5, lw=1.2):
    kde  = gaussian_kde(pc.T)
    pad  = (pc.max(axis=0) - pc.min(axis=0)) * 0.15
    lo, hi = pc.min(axis=0) - pad, pc.max(axis=0) + pad
    xx, yy = np.mgrid[lo[0]:hi[0]:80j, lo[1]:hi[1]:80j]
    z = kde(np.vstack([xx.ravel(), yy.ravel()])).reshape(xx.shape)
    ax.contour(xx, yy, z, levels=levels, colors=[color], alpha=0.65, linewic

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# ── Plot 1: Train (blue △) vs Test (crimson ×) ──────────────────────────
_contour(ax1, train_pc, C1)
```

```
_contour(ax1, OOS_pc,   C2)
ax1.scatter(*_sub(train_pc, seed=0).T, c=C1, marker='^', s=22, alpha=0.45, l
ax1.scatter(*_sub(OOS_pc,   seed=1).T, c=C2, marker='x', s=28, alpha=0.55,
            label='Test (OOS)', linewidths=0.8)
ax1.set_xlabel(xlab); ax1.set_ylabel(ylab)
ax1.set_title('Train vs Test', fontweight='bold')
ax1.legend(fontsize=9, markerscale=1.5)
ax1.grid(True, alpha=0.25)

# ── Plot 2: Generated (crimson ×) vs Baselines (blue △ …) ──────────────
_contour(ax2, gen_pc, C2)
ax2.scatter(*_sub(gen_pc, seed=0).T, c=C2, marker='x', s=28, alpha=0.45,
            label='Generated', linewidths=0.8)
for i, (bname, bpc, bc) in enumerate(base_pcs):
    _contour(ax2, bpc, bc)
    ax2.scatter(*_sub(bpc, seed=i+1).T, c=bc, marker=MARKERS[i % len(MARKERS
                s=22, alpha=0.45, label=bname)
ax2.set_xlabel(xlab); ax2.set_ylabel(ylab)
ax2.set_title('Generated vs Baselines', fontweight='bold')
ax2.legend(fontsize=9, markerscale=1.5)
ax2.grid(True, alpha=0.25)

fig.suptitle('2.6 PCA Projection (fitted on training set)', fontsize=13, for
plt.tight_layout()
plt.show()
```



In [63]:
```
jupyter nbconvert --to webpdf evaluation.ipynb
```

```
  Cell In[63], line 1
    jupyter nbconvert --to webpdf evaluation.ipynb
             ^
SyntaxError: invalid syntax
```